Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026

Week 8 - Solr

Objective 1 – Conceptual Foundation

Apache Solr is an open-source search platform designed for fast, scalable full-text search and indexing. Traditional databases work well when you already know which rows you want, but they are not optimized for searching large volumes of text. Solr solves this by organizing text specifically for search. Applications send documents to Solr through REST APIs, Solr indexes them, and clients later submit queries to retrieve matching results. Solr also supports sharding and replication, allowing data to be distributed across multiple servers for scalability and fault tolerance. ZooKeeper is used to coordinate the cluster, manage configuration, and track which nodes are active.

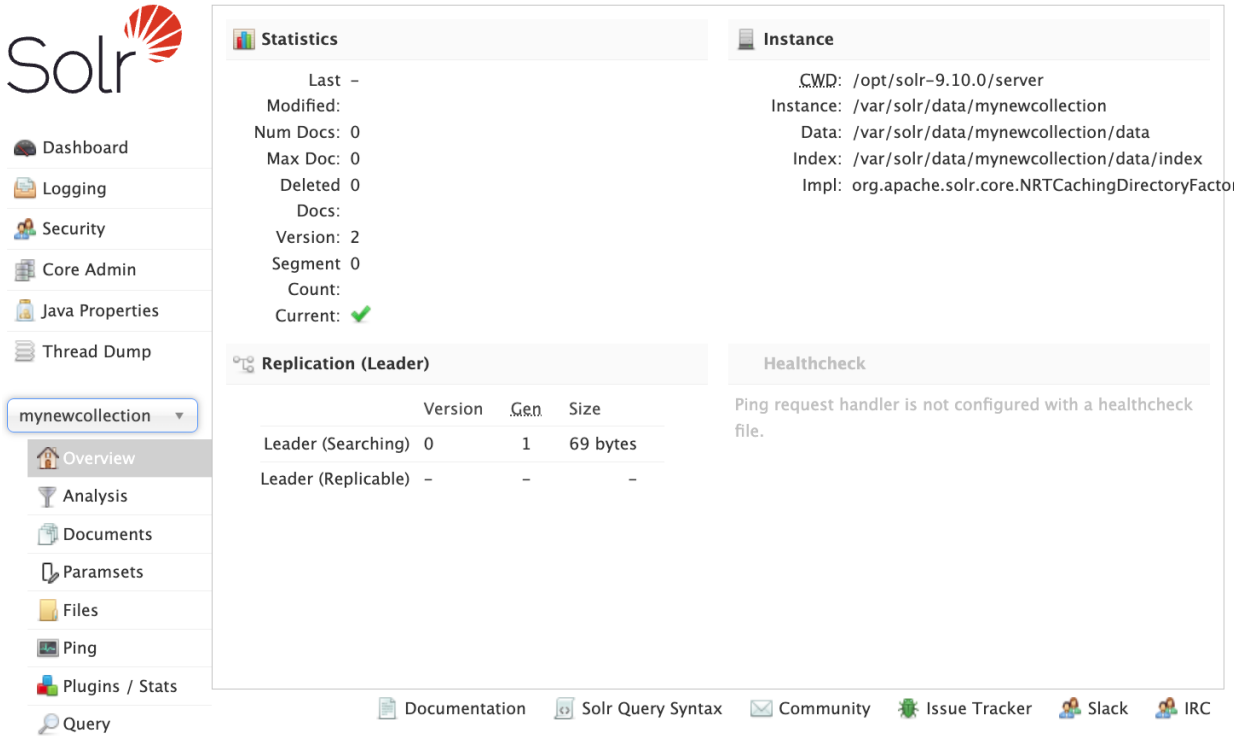
In a typical workflow, applications send documents to Solr, which processes and indexes them. When a query is made, Solr searches across the relevant shards, merges the results, and returns them to the application. This distributed architecture allows Solr to continue operating even if individual nodes fail. These features make Solr suitable for real-world production systems where reliability and performance matter.

At the core of Solr is the inverted index. Unlike a traditional index, which maps documents to their contents, an inverted index maps terms to the documents that contain them. When Solr ingests documents, it analyzes the text by tokenizing it, lowercasing, and removing stop words. The resulting terms are stored in the inverted index along with

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026
document IDs and word positions. For example, the word “cat” might map to documents 1, 3, and 6, along with its positions in each document. Because of this reverse lookup structure, Solr does not need to scan every document during a search — it can directly locate matching documents, which is what makes full-text queries extremely fast.

Objective 2 – Creating a Solr Collection

The collection is active and marked as current, indicating it initialized without errors. The instance path, data directory, and index directory are all established, confirming that the collection was successfully created and loaded by Solr.



Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026

Objective 3 – Adding Data to the Collection

Two product records are written to a JSON file and then posted to the mynewcollection core using Solr's indexing tool. Solr accepts the file, indexes one batch successfully, and commits the changes, making the new documents immediately searchable.

```
Created new core 'mynewcollection'
solr@4220785c99b4:/opt/solr-9.10.0$ echo '[
  {"id":"1", "name":"Product A", "category":"Electronics", "price":"100"},
  {"id":"2", "name":"Product B", "category":"Books", "price":"20"}
]' > /tmp/products.json
solr@4220785c99b4:/opt/solr-9.10.0$ /opt/solr/bin/solr post -c mynewcollection /tmp
/products.json
Posting files to [base] url http://localhost:8983/solr/mynewcollection/update...
Entering auto mode. File endings considered are xml,json,jsonl,csv,pdf,doc,docx,ppt
,pptx,xls,xlsx,odt,odp,ods,ott,otp,ots,rtf,htm,html,txt,log
POSTing file products.json (application/json) to [base]/json/docs
1 files indexed.
COMMITting Solr index changes to http://localhost:8983/solr/mynewcollection/update.
..
WARNING: URLs provided to this tool needn't include Solr's context-root (e.g. "/sol
r"). Such URLs are deprecated and support for them will be removed in a future rele
ase. Correcting from [http://localhost:8983/solr] to [http://localhost:8983/].
Time spent: 0:00:00.840
solr@4220785c99b4:/opt/solr-9.10.0$
```

Objective 4 – Querying the Collection

A query is executed against the collection requesting all documents, and the response returns a successful status with two records found. Both previously indexed products

appear in the results, confirming the documents were stored correctly and are retrievable

from the collection.

```
solr@4220785c99b4:/opt/solr-9.10.0$ curl "http://localhost:8983/solr/mynewcollectio
n/select?q=*:*"
{
  "responseHeader":{
    "status":0,
    "QTime":3,
    "params":{
      "q":"*:*"
    }
  },
  "response":{
    "numFound":2,
    "start":0,
    "numFoundExact":true,
    "docs":[{
      "id":"1",
      "name":["Product A"],
      "category":["Electronics"],
      "price":[100],
      "_version_":1857237595789983744,
      "_root_":"1"
    },{
      "id":"2",
      "name":["Product B"],
      "category":["Books"],
      "price":[20],
      "_version_":1857237595893792768,
      "_root_":"2"
    }]
  }
}solr@4220785c99b4:/opt/solr-9.10.0$
```

A filtered query is executed against the collection using the category:Electronics

parameter, and Solr returns exactly one matching document. The response confirms that

only Product A meets the category criteria, demonstrating that field-based searching is

working correctly.

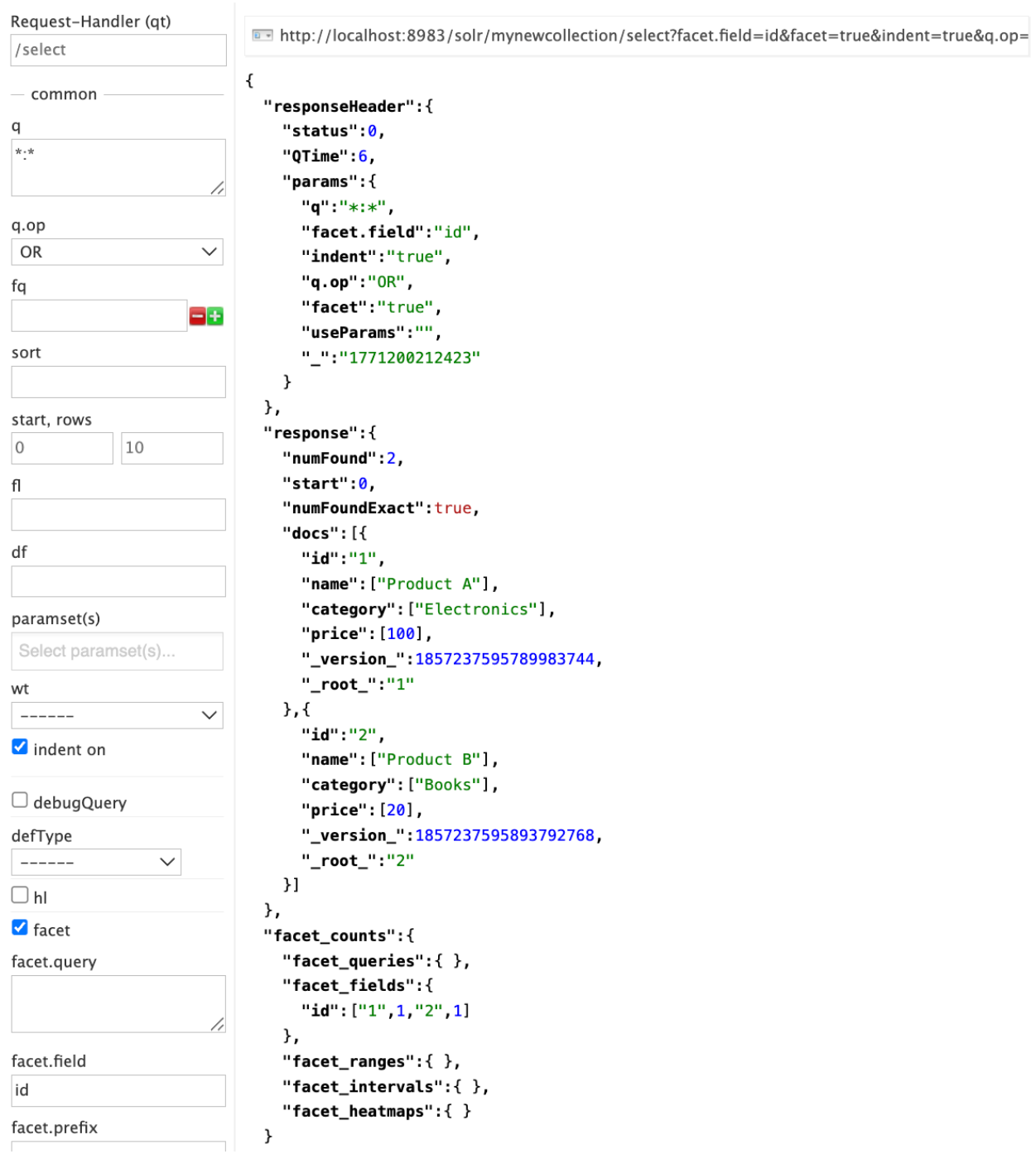
Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026

```
}solr@4220785c99b4:/opt/solr-9.10.0$ curl "http://localhost:8983/solr/mynewcollecti
on/select?q=category:Electronics"
{
  "responseHeader":{
    "status":0,
    "QTime":22,
    "params":{
      "q":"category:Electronics"
    }
  },
  "response":{
    "numFound":1,
    "start":0,
    "numFoundExact":true,
    "docs":[{
      "id":"1",
      "name":["Product A"],
      "category":["Electronics"],
      "price":[100],
      "_version_":1857237595789983744,
      "_root_":"1"
    }]
  }
}solr@4220785c99b4:/opt/solr-9.10.0$
```

Objective 5 – Advanced Querying in the Solr Web Interfaces

A faceted query is executed on the id field while matching all documents, returning both records along with per-ID counts. The facet results show each document ID appears once, confirming that two unique documents exist in the collection and that Solr's faceting is correctly aggregating field values.

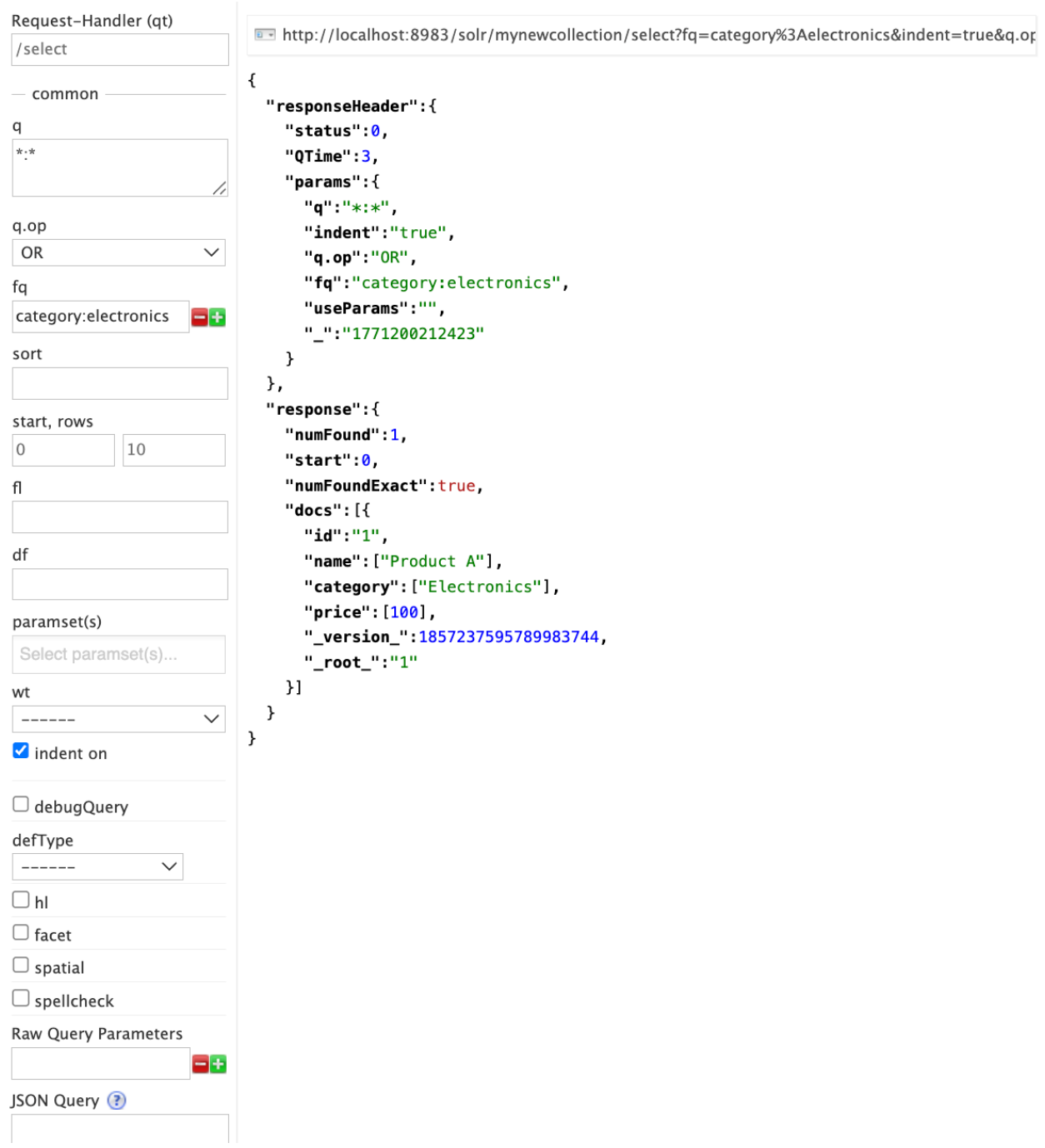
Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026




```
{
  "responseHeader":{
    "status":0,
    "QTime":6,
    "params":{
      "q":"*:*",
      "facet.field":"id",
      "indent":"true",
      "q.op":"OR",
      "facet":"true",
      "useParams":"",
      "_":"1771200212423"
    }
  },
  "response":{
    "numFound":2,
    "start":0,
    "numFoundExact":true,
    "docs":[{
      "id":"1",
      "name":["Product A"],
      "category":["Electronics"],
      "price":[100],
      "_version_":1857237595789983744,
      "_root_":"1"
    },{
      "id":"2",
      "name":["Product B"],
      "category":["Books"],
      "price":[20],
      "_version_":1857237595893792768,
      "_root_":"2"
    }]
  },
  "facet_counts":{
    "facet_queries":{ },
    "facet_fields":{
      "id":["1",1,"2",1]
    },
    "facet_ranges":{ },
    "facet_intervals":{ },
    "facet_heatmaps":{ }
  }
```

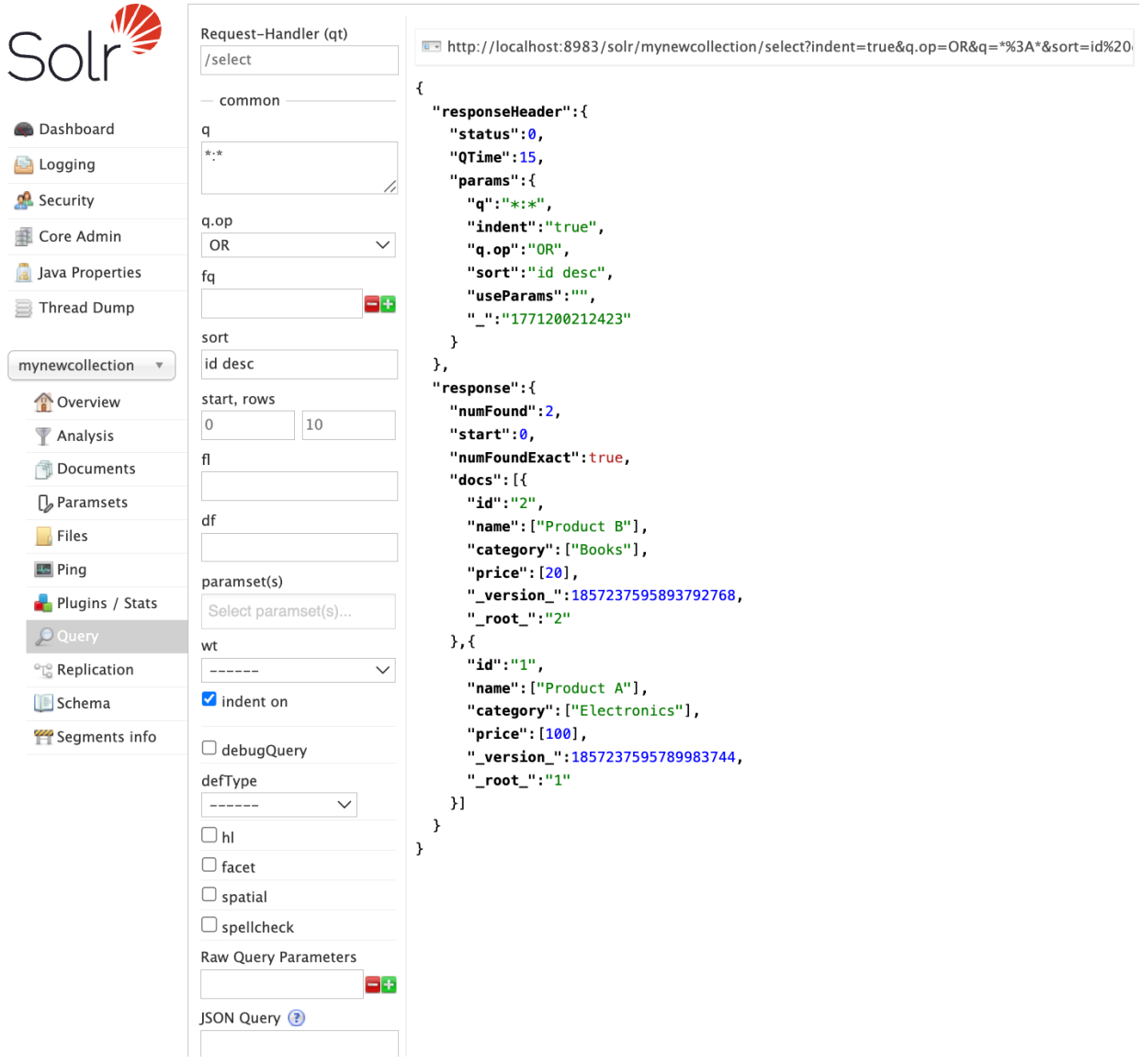
Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026

A filter query is applied on the category field for electronics, narrowing the results to matching documents only. The response returns a single record, confirming that the filter correctly restricts results to Product A and excludes non-matching categories.



Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026

A sort is applied on the id field in descending order while returning all documents in the collection. The results are reordered so Product B (id 2) appears first, followed by Product A (id 1), confirming that server-side sorting is working correctly.



Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026

Objective 6 – Working with Custom JSON Data

A new core named mycollection is created, then a JSON dataset containing five product records is written to /tmp/sample.json after correcting a permissions issue. The file is posted to Solr's update endpoint and committed successfully, indexing all five documents and making them immediately available for querying.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026

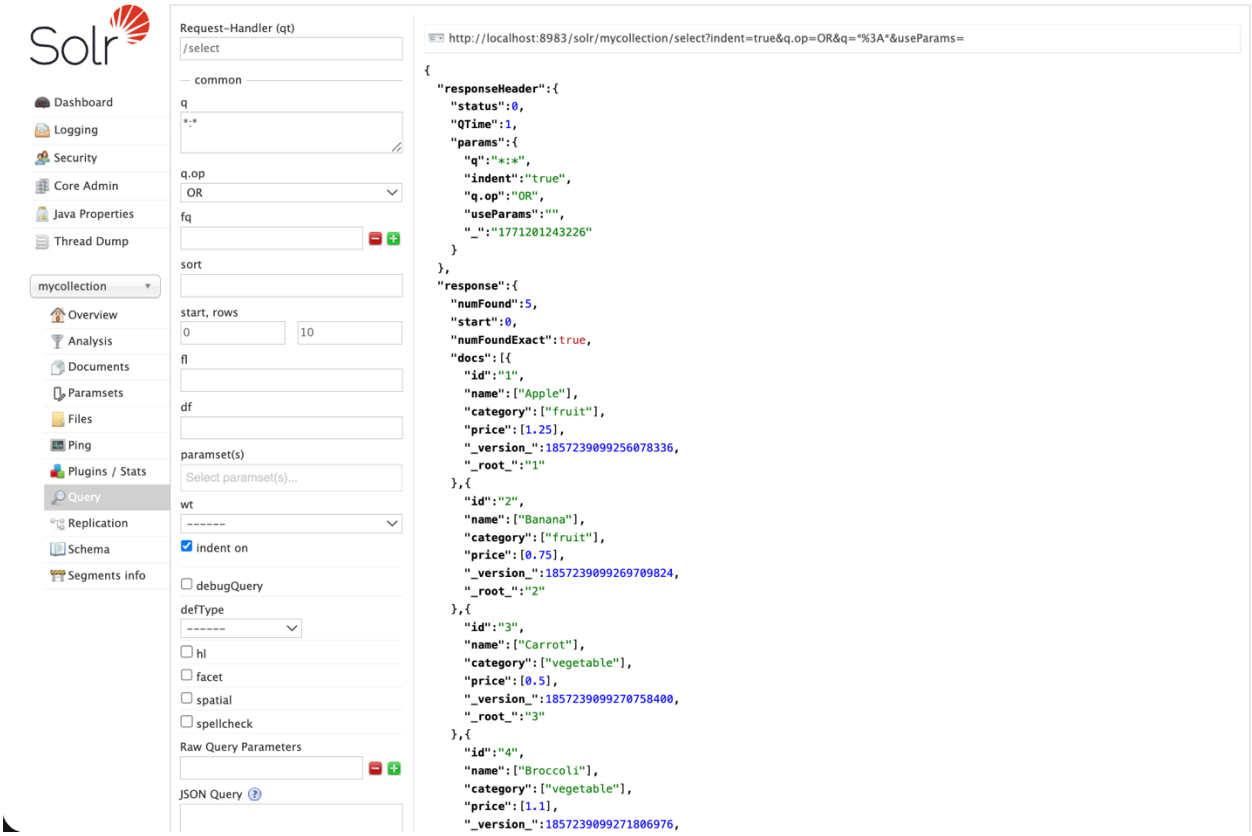
```
solr@4220785c99b4:/opt/solr-9.10.0$ solr create -c mycollection
Neither --zk-host or --solr-url parameters provided so assuming solr url is http://localh
ost:8983.
WARNING: Using _default configset. Data driven schema functionality is enabled by default
, which is
         NOT RECOMMENDED for production use.

         To turn it off:
            curl http://localhost:8983/solr/mycollection/config -d '{"set-user-property":
 {"update.autoCreateFields":"false"}}'
         Or:
            bin/solr config -c mycollection -s http://localhost:8983 --action set-user-pr
operty --property update.autoCreateFields --value false

Created new core 'mycollection'
solr@4220785c99b4:/opt/solr-9.10.0$ cat > sample.json <<EOF
[
  {"id":"1","name":"Apple","category":"fruit","price":1.25},
  {"id":"2","name":"Banana","category":"fruit","price":0.75},
  {"id":"3","name":"Carrot","category":"vegetable","price":0.50},
  {"id":"4","name":"Broccoli","category":"vegetable","price":1.10},
  {"id":"5","name":"Steak","category":"meat","price":9.99}
]
EOF
bash: sample.json: Permission denied
solr@4220785c99b4:/opt/solr-9.10.0$ cd /tmp/
solr@4220785c99b4:/tmp$ cat > sample.json <<EOF
[
  {"id":"1","name":"Apple","category":"fruit","price":1.25},
  {"id":"2","name":"Banana","category":"fruit","price":0.75},
  {"id":"3","name":"Carrot","category":"vegetable","price":0.50},
  {"id":"4","name":"Broccoli","category":"vegetable","price":1.10},
  {"id":"5","name":"Steak","category":"meat","price":9.99}
]
EOF
solr@4220785c99b4:/tmp$ curl http://localhost:8983/solr/mycollection/update?commit=true \
  -H "Content-Type: application/json" \
  --data-binary @sample.json
{
  "responseHeader":{
    "status":0,
    "QTime":199
  }
solr@4220785c99b4:/tmp$
```

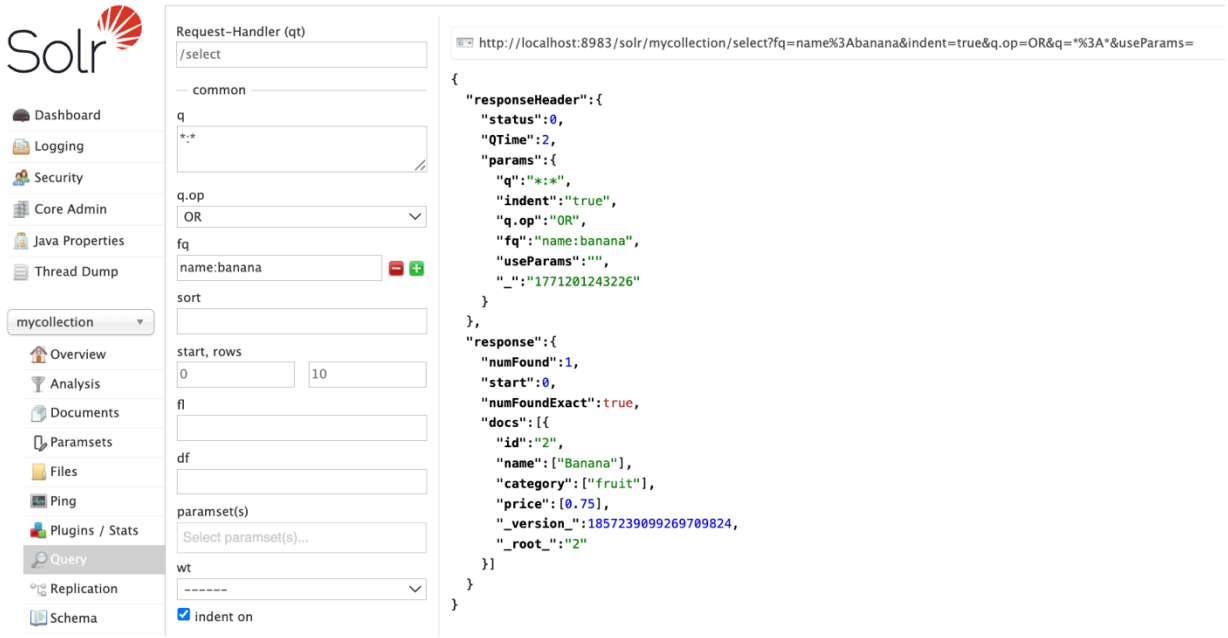
Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026

A wildcard query is executed against the mycollection core, returning all indexed documents. The response reports five total matches and lists each product record, confirming that the full dataset was ingested successfully and is available for search.



A filter query is applied on the `name` field for `banana`, reducing the result set to a single matching document. The response returns only the Banana record, confirming that field-level filtering is functioning correctly and that individual items can be retrieved by name.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026



A filter query is applied on the `category` field for `vegetable`, limiting results to matching documents only. The response returns two records, Carrot and Broccoli, confirming that multiple documents within the same category are correctly retrieved.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 15, 2026

